

Билет 13. Future и Promise. Асинхронное выполнение задач. Callback hell. Continuations, chaining, combining

0. Формулировка билета

Примитивы Future и Promise. Применение для асинхронного выполнения задач. Callback hell. Future continuations, chaining и combining.

Главная идея:

Future — это представление результата, который появится в будущем.

Promise — это объект, через который производитель устанавливает этот результат.

Вместе они позволяют запустить задачу асинхронно, сразу вернуть вызывающему коду “будущий результат”, а потом заполнить его значением или ошибкой.

Ключевая схема:

```
producer thread:
  promise.set_value(result)
      |
      ▼
  shared state
      ▲
      |
consumer thread:
  future.get() / continuation
```

1. Где мы находимся в истории сервера

До этого мы прошли несколько архитектур.

Thread pool + blocking I/O

```
accept client
submit task to thread pool
worker делает blocking read/write
```

Проблема:

медленный клиент может занять worker
если все workers заблокированы, новые клиенты ждут

Однопоточный `epoll`

один I/O-thread
`epoll_wait`
non-blocking read/write

Плюс:

один поток может обслуживать много соединений

Но есть новая проблема.

Если обработчик делает CPU-heavy работу:

```
bool result = is_prime(very_big_number);
```

то единственный I/O-thread занят вычислением.

Пока он считает:

новые события `epoll` не обрабатываются
другие клиенты ждут
latency растёт

Вывод:

Нужно совместить `epoll` для I/O и thread pool для тяжёлой CPU-работы.

2. Гибридная архитектура: I/O-loop + CPU thread pool

Идея:

I/O-thread:
 `epoll`
 accept/read/write
 не делает тяжёлые вычисления

CPU thread pool:

считает тяжёлые задачи

future/promise:

связывает отправку задачи и получение результата

Схема:

```
client sends number
  |
  ▼
I/O-thread reads request
  |
  ▼
submit is_prime(number) to CPU pool
  |
  ▼
worker computes result
  |
  ▼
future becomes ready
  |
  ▼
I/O-thread sends response
```

Важно:

I/O-thread не должен блокироваться на `future.get()`, иначе мы снова заблокируем event loop.

3. Что такое Future

Future — это объект, представляющий результат, который станет доступен позже.

Он может уметь:

```
future.wait();      // дождаться готовности
future.get();       // получить результат
future.wait_for();  // проверить с timeout
```

Смысл:

у меня ещё нет значения,
но оно появится в будущем

Пример:

```
std::future<int> f = submit_to_pool([] {
    return 42;
});

// позже
int x = f.get();
```

`future.get()` обычно блокирует поток, если результат ещё не готов.

4. Что такое Promise

Promise — это “пишущий конец” пары.

Он устанавливает результат:

```
promise.set_value(value);
```

или ошибку:

```
promise.set_exception(std::current_exception());
```

Пара создаётся так:

```
std::promise<int> p;
std::future<int> f = p.get_future();
```

Обычно:

```
future отдаётся потребителю
promise остаётся у производителя
```

Схема:

```
std::promise<int> p;
std::future<int> f = p.get_future();

std::thread worker([p = std::move(p)]() mutable {
    try {
        int result = compute();
        p.set_value(result);
    } catch (...) {
        p.set_exception(std::current_exception());
    }
})
```

```
});  
  
int value = f.get();
```

5. Shared state

`Future` и `Promise` связаны через **shared state**.

Упрощённо shared state содержит:

```
флаг готовности  
значение или exception_ptr  
mutex  
condition_variable
```

Схема:

```
promise → shared state ← future
```

`promise.set_value(x)`:

```
кладёт значение в shared state  
помечает ready = true  
будит ожидающих
```

`future.get()`:

```
ждёт ready == true  
если значение — возвращает его  
если exception — перебрасывает исключение
```

Важная фраза:

`Future` и `Promise` не передают значение напрямую друг другу. Они оба смотрят на общий shared state.

6. One-shot семантика

`Future/Promise` — это обычно **one-shot** канал.

То есть:

promise устанавливает значение один раз
future забирает значение один раз

Нельзя несколько раз сделать:

```
p.set_value(...);
```

И обычный `std::future` нельзя нормально копировать для многих потребителей.

Если нужно несколько потребителей, используется:

```
std::shared_future<T>
```

Он копируемый, и несколько потоков могут ждать один и тот же результат.

7. Применение для асинхронного выполнения задач

Допустим, у нас есть thread pool, который принимает `void()`-задачи.

Чтобы получить результат, задачу можно обернуть в `std::packaged_task`.

```
template <typename F>
auto submit(ThreadPool& pool, F f) -> std::future<decltype(f())> {
    using R = decltype(f());

    auto task = std::make_shared<std::packaged_task<R()>>(std::move(f));

    std::future<R> future = task->get_future();

    pool.submit([task] {
        (*task)();
    });

    return future;
}
```

Использование:

```
auto future = submit(pool, [] {
    return is_prime(1000000007);
});
```

```
// можно дождаться позже  
bool prime = future.get();
```

Что делает `packaged_task`:

```
хранит callable  
при вызове выполняет callable  
результат кладёт в shared state  
future получает этот результат
```

8. Почему просто `future.get()` плохо для event loop

В обычном коде можно написать:

```
bool result = future.get();
```

Но если это делает I/O-thread, будет плохо.

```
I/O-thread вызвал get()  
future ещё не готов  
I/O-thread заблокировался  
epoll_wait больше не вызывается  
сервер перестал обслуживать клиентов
```

То есть:

В event-driven архитектуре нельзя блокировать I/O-loop на `future.get()`.

Нужен другой механизм:

```
когда future готова,  
выполнить callback / continuation
```

9. Наивный polling futures

Можно было бы хранить список futures:

```
std::vector<std::future<Result>> pending;
```

и периодически проверять:

```
for (auto& f : pending) {  
    if (f.wait_for(0ms) == std::future_status::ready) {  
        process(f.get());  
    }  
}
```

Но это плохой вариант.

Проблема:

```
если новых epoll-событий нет,  
I/O-thread спит в epoll_wait,  
check_pending() не вызывается,  
хотя future уже могла стать готовой
```

То есть результат готов, а I/O-loop об этом не узнаёт.

Решение:

```
нужно событие, которое разбудит I/O-thread
```

Например:

```
eventfd
```

Worker после вычисления пишет в `eventfd`, `eventfd` зарегистрирован в `epoll`, и I/O-thread просыпается.

10. Callback

Callback — функция, которую нужно вызывать, когда результат станет готов.

Вместо:

```
auto result = future.get();  
process(result);
```

ХОТИМ:


```
future.Subscribe([](Result result) {  
    process(result);  
});
```

Смысл:

не блокируй текущий поток
когда результат будет готов – вызови этот код

Так мы переходим от blocking wait к асинхронной реакции.

11. Реализация Subscribe: гонка результата и callback

Есть два события:

1. producer вызвал set_value
2. consumer вызвал Subscribe(callback)

Они могут произойти в любом порядке.

Случай 1. Callback подписался первым

Subscribe(callback)
результата ещё нет
callback сохраняется

позже set_value(result)
вызывает callback(result)

Случай 2. Результат пришёл первым

set_value(result)
callback ещё нет
результат сохраняется

позже Subscribe(callback)
сразу вызывает callback(result)

Нужно корректно обработать оба случая.

12. Автомат состояний для Subscribe

Shared state можно представить как автомат:

```
EMPTY          — нет ни результата, ни callback
ONLY_RESULT    — результат есть, callback ещё нет
ONLY_CALLBACK  — callback есть, результата ещё нет
DONE           — callback вызван
```

Схема:

```
Subscribe первым:
    EMPTY → ONLY_CALLBACK

set_value потом:
    ONLY_CALLBACK → DONE
    вызвать callback

set_value первым:
    EMPTY → ONLY_RESULT

Subscribe потом:
    ONLY_RESULT → DONE
    вызвать callback
```

Ключевая мысль:

`Subscribe` и `set_value` соревнуются. Кто приходит первым — сохраняет свою часть. Кто приходит вторым — видит, что вторая часть уже есть, и вызывает callback.

13. Упрощённый код Subscribe / set_value

```
enum Tag {
    EMPTY,
    ONLY_RESULT,
    ONLY_CALLBACK,
    DONE
};

template <typename T>
struct SharedState {
    std::atomic<int> tag{EMPTY};

    std::optional<T> result;
    std::function<void(T)> callback;
};
```

```

template <typename T>
class MyFuture {
public:
    explicit MyFuture(std::shared_ptr<SharedState<T>> s)
        : state_(std::move(s))
    {}

    void Subscribe(std::function<void(T)> cb) {
        state_->callback = std::move(cb);

        int expected = EMPTY;

        if (state_->tag.compare_exchange_strong(
            expected,
            ONLY_CALLBACK
        )) {
            return;
        }

        // Если мы здесь, значит результат уже был.
        // В учебной версии считаем expected == ONLY_RESULT.
        state_->callback(std::move(*state_->result));
        state_->tag.store(DONE);
    }

private:
    std::shared_ptr<SharedState<T>> state_;
};

template <typename T>
class MyPromise {
public:
    explicit MyPromise(std::shared_ptr<SharedState<T>> s)
        : state_(std::move(s))
    {}

    void set_value(T value) {
        state_->result = std::move(value);

        int expected = EMPTY;

        if (state_->tag.compare_exchange_strong(
            expected,
            ONLY_RESULT
        )) {
            return;
        }

        // Если мы здесь, значит callback уже был.
        // В учебной версии считаем expected == ONLY_CALLBACK.

```

```

        state_->callback(std::move(*state_->result));
        state_->tag.store(DONE);
    }

private:
    std::shared_ptr<SharedState<T>> state_;
};

```

В реальном коде нужно аккуратно работать с:

```

memory_order
исключениями
разрушением shared state
повторным set_value
потокобезопасностью result/callback

```

Но идея такая:

`Subscribe` и `set_value` синхронизируются через атомарный тег.

14. Callback hell

Если использовать только callbacks, последовательные асинхронные шаги превращаются во вложенность.

Например:

```

FooAsync(input).Subscribe([=](Output o) {
    AsyncA(o).Subscribe([=](OutputA a) {
        AsyncB(a).Subscribe([=](OutputB b) {
            AsyncC(b).Subscribe([=](OutputC c) {
                process(c);
            });
        });
    });
});

```

Это называется **callback hell**.

Проблемы:

```

код уезжает вправо
сложно читать
сложно обрабатывать ошибки
сложно делать early return

```

сложно комбинировать параллельные операции
сложно контролировать, в каком потоке выполняется callback

Нужен способ писать цепочку линейно.

15. Continuation

Continuation — это продолжение вычисления.

То есть:

когда future готова,
выполни следующий шаг

Например:

```
Future<int> f = GetNumberAsync();  
  
Future<std::string> g = f.thenValue([](int x) {  
    return std::to_string(x);  
});
```

Смысл:

f станет готова с int
thenValue применит функцию `int → string`
g станет готова со string

Continuation превращает:

future<T>

в:

future<U>

16. Chaining

Chaining — построение цепочки continuations.

Вместо callback hell:

```

FooAsync(input).Subscribe(=[](Output o) {
    AsyncA(o).Subscribe(=[](OutputA a) {
        AsyncB(a).Subscribe(=[](OutputB b) {
            AsyncC(b).Subscribe(=[](OutputC c) {
                process(c);
            });
        });
    });
});

```

пишем:

```

FooFuture(input)
    .thenValue(AsyncA)
    .thenValue(AsyncB)
    .thenValue(AsyncC)
    .thenValue(=[](OutputC c) {
        return process(c);
    });

```

Код становится похож на обычную последовательность:

```

Foo
then A
then B
then C
then process

```

17. `thenValue`: синхронный вариант

Пусть функция обычная:

```

U func(T value);

```

Тогда:

```

Future<U> thenValue(func);

```

работает так:

```

когда исходная Future<T> готова,
берём T

```

вызываем `U func(T)`
кладём `U` в новую `Promise<U>`
возвращаем `Future<U>`

Схема:

```
Future<T>
  |
  ▼
thenValue(T -> U)
  |
  ▼
Future<U>
```

Пример:

```
auto f =
  ReadNumberAsync()
  .thenValue([](int x) {
    return x * 2;
  })
  .thenValue([](int x) {
    return std::to_string(x);
  });
```

18. `thenValue`: асинхронный вариант

Иногда следующий шаг сам асинхронный:

```
Future<User> GetUserAsync(Id id);
```

Если применить обычный `map`, получилось бы:

```
Future<Future<User>>
```

Это неудобно.

Поэтому нужен асинх- `thenValue`, который делает `flatten`:

```
Future<T>
  |
  ▼
thenValue(T -> Future<U>)
```

|
▼
Future<U>

То есть он склеивает вложенную future.

Это также называют:

```
flatMap  
bind  
thenCompose
```

Пример:

```
GetUserIdAsync()  
    .thenValue([](UserId id) {  
        return GetUserAsync(id); // возвращает Future<User>  
    })  
    .thenValue([](User user) {  
        return user.name;  
    });
```

19. Ошибки: `set_exception`, `thenError`

Вычисление может завершиться ошибкой.

`Promise` может сделать:

```
promise.set_exception(std::current_exception());
```

`Future.get()` потом перебросит исключение.

В цепочках нужны обработчики ошибок:

```
future  
    .thenValue([](Value v) {  
        return process(v);  
    })  
    .thenError([](std::exception_ptr e) {  
        return fallback();  
    });
```

Идея:


```
thenValue – успешный путь  
thenError – путь ошибки
```

В функциональном стиле это похоже на:

```
result<T, E>  
expected<T, E>
```

20. Combining futures

Chaining — это последовательная композиция:

```
A → B → C
```

Но часто нужны параллельные операции:

```
A и B и C одновременно
```

Для этого есть combining.

CollectAll

Ждёт все future.

```
auto f1 = Rpc1();  
auto f2 = Rpc2();  
auto f3 = Rpc3();  
  
auto all = collectAll(f1, f2, f3);
```

Смысл:

```
all станет готова, когда готовы f1, f2, f3
```

Результат:

```
tuple/result vector со всеми результатами
```

Используется, когда нужны все ответы.

CollectAny

Ждёт первую завершившуюся future.

```
auto first = collectAny(f1, f2, f3);
```

Смысл:

first станет готова, когда завершится первая из f1, f2, f3

Важно:

первая завершившаяся может быть и успешной, и ошибочной

CollectAnyWithoutException

Ждёт первую успешную future.

если первая завершилась ошибкой,
ждём следующую,
пока не будет успеха или пока все не упадут

Используется для:

replicated requests
fallback
hedging

21. Timeout через combining

Timeout удобно выразить через “первая из двух future”.

```
request_future  
timer_future
```

Комбинатор:

```
auto result = first(request_future, timer_future);
```

Смысл:

если запрос успел раньше timeout – берём ответ
если timer сработал раньше – считаем timeout

Это важная сила futures:

Они хорошо выражают параллельную композицию и гонку нескольких асинхронных действий.

22. Executors

Проблема continuations:

В каком потоке выполнится callback?

Если результат уже готов при подписке, continuation может выполниться в потоке подписчика.

Если результат будет готов позже, continuation может выполниться в потоке producer-а.

Это недетерминированно.

Например, тяжёлая continuation может случайно выполниться в I/O-thread и заблокировать event loop.

Нужно явно задавать место исполнения.

23. Executor

Executor — абстракция “где и как выполнить функцию”.

Интерфейс:

```
struct IExecutor {  
    virtual void Execute(std::function<void()> f) = 0;  
};
```

Типы executors:

InlineExecutor – выполнить сразу в текущем потоке
ThreadPoolExecutor – положить задачу в thread pool

SerializedExecutor / Strand – выполнить задачи последовательно
ManualExecutor – ручной прогон, удобно для тестов

Future может иметь метод:

```
via(executor)
```

Смысл:

все следующие continuations выполняй на этом executor-e

24. `via`

Пример:

```
SomeRpc(input)
    .via(&cpuExecutor)
    .thenValue([](Response r) {
        return heavyComputation(r);
    })
    .via(&ioExecutor)
    .thenValue([](Result result) {
        writeResponse(result);
    });
```

Смысл:

CPU-heavy работа – в CPU thread pool
I/O работа – обратно в I/O-thread

Это решает проблему:

случайный поток исполнения callback-a

и позволяет разделить нагрузки.

25. Гибрид `epoll` + `futures` + `eventfd`

Полная архитектура:

```
I/O-thread:
    epoll
    server socket
    eventfd
    client fds

CPU thread pool:
    тяжёлые вычисления

futures:
    композиция вычислений и возврат результата
```

Алгоритм:

1. I/O-thread прочитал число от клиента.
2. Создаём future-цепочку.
3. CPU-heavy шаг отправляем на cpuExecutor.
4. Когда результат готов, continuation переключается на ioExecutor.
5. ioExecutor кладёт задачу в очередь I/O-thread и пишет в eventfd.
6. epoll просыпается по eventfd.
7. I/O-thread забирает готовую задачу, кладёт ответ в output_buffer и включает EPOLLOUT.
8. Когда fd готов на запись, handle_write отправляет ответ.

Пример цепочки:

```
makeFuture(number)
    .via(&cpuExecutor)
    .thenValue(is_prime)
    .via(&ioExecutor)
    .thenValue([fd, this])(bool prime) {
        enqueue_write(fd, prime ? "prime\n" : "not prime\n");
    };
```

Что достигнуто:

```
I/O-thread не считает CPU-heavy
I/O-thread не блокируется на future.get()
worker pool не пишет напрямую в client_fd
результаты приходят в I/O-thread через eventfd
```

26. Future как значение

Более высокий взгляд:

Future — это не просто “штука с get”.
Future — это будущее значение.

Обычное значение:

```
int x;
```

можно передать в функцию:

```
f(x);
```

Будущее значение:

```
Future<int> fx;
```

можно преобразовывать:

```
fx.thenValue(f);
```

То есть:

```
int          → обычное значение  
Future<int> → значение, которое появится позже
```

27. Data flow vs control flow

В потоках, фиберах и корутинах мы часто думаем про **control flow**:

```
сначала сделай A  
потом сделай B  
потом сделай C
```

Во futures мы думаем про **data flow**:

```
когда появится значение A,  
передай его в B,  
потом в C,  
а параллельно собери результаты X и Y
```

Futures строят внутри C++ маленький асинхронный язык:

```
constructors: makeFuture, spawn, value
combinators: thenValue, flatMap, collectAll, collectAny
terminators: get, detach
```

28. Что сказать на экзамене: короткая версия

Можно отвечать так:

`Future` — это объект, представляющий результат, который станет доступен позже.
`Promise` — это объект-производитель, который устанавливает значение или исключение в общий shared state. `future` и `promise` образуют one-shot канал: `promise` один раз делает `set_value` или `set_exception`, а `future` потом получает результат через `get` или продолжение.

Для асинхронного выполнения задачу можно положить в thread pool, а вызывающему сразу вернуть future. Обычно это делается через `std::packaged_task`: задача выполняется в worker-е, результат попадает в shared state, а вызывающий код держит future.

В event loop нельзя блокироваться на `future.get()`, потому что I/O-thread перестанет обрабатывать `epoll`. Поэтому нужны callbacks/continuations: future получает метод `Subscribe` или `thenValue`, который выполняет продолжение, когда результат готов.

Голые callbacks приводят к callback hell: вложенные асинхронные вызовы становятся плохо читаемыми и плохо обрабатывают ошибки. Continuations решают это: `future.thenValue(A).thenValue(B).thenValue(C)` превращает вложенность в линейную цепочку. Если следующий шаг сам возвращает future, нужен `async-then` или `flatMap`, который склеивает `Future<Future<T>>` в `Future<T>`.

Для параллельной композиции используют комбинаторы: `CollectAll` ждёт все future, `CollectAny` ждёт первую завершившуюся, `CollectAnyWithoutException` — первую успешную. Через `CollectAny` удобно выражать timeout: гоняем request future и timer future.

Важная проблема continuations — неизвестно, в каком потоке они выполнятся. Поэтому вводят executors: абстракцию “где выполнить код”. Метод `via(executor)` переключает следующие продолжения на нужный executor: CPU-heavy — в thread pool, I/O — обратно в I/O-thread. В гибридном сервере I/O-thread читает запрос, CPU-pool считает, а результат через future continuation и eventfd возвращается в I/O-thread для записи ответа.

29. Мини-код для ответа на доске

Future/Promise

```
std::promise<int> p;
std::future<int> f = p.get_future();

std::thread worker([p = std::move(p)]() mutable {
    try {
        int result = compute();
        p.set_value(result);
    } catch (...) {
        p.set_exception(std::current_exception());
    }
});

int value = f.get();
```

Submit в thread pool с future

```
template <typename F>
auto submit(ThreadPool& pool, F f) -> std::future<decltype(f())> {
    using R = decltype(f());

    auto task =
        std::make_shared<std::packaged_task<R()>>(std::move(f));

    std::future<R> future = task->get_future();

    pool.submit([task] {
        (*task)();
    });

    return future;
}
```

Callback hell

```
FooAsync(input).Subscribe([=](Output o) {
    AsyncA(o).Subscribe([=](OutputA a) {
        AsyncB(a).Subscribe([=](OutputB b) {
            AsyncC(b).Subscribe([=](OutputC c) {
                process(c);
            });
        });
    });
});
```



```
});  
});
```

Chaining

```
FooFuture(input)  
  .thenValue(AsyncA)  
  .thenValue(AsyncB)  
  .thenValue(AsyncC)  
  .thenValue([](OutputC c) {  
    return process(c);  
  });
```

Executors

```
SomeRpc(input)  
  .via(&cpuExecutor)  
  .thenValue([](Response r) {  
    return heavyComputation(r);  
  })  
  .via(&ioExecutor)  
  .thenValue([](Result result) {  
    writeResponse(result);  
  });
```

30. Мини-проверка

1. Что такое `Future` ?
2. Что такое `Promise` ?
3. Что такое shared state между future и promise?
4. Почему future/promise обычно one-shot?
5. Как через `packaged_task` вернуть результат из thread pool?
6. Почему в event loop нельзя делать `future.get()` ?
7. Что такое callback hell?
8. Что такое continuation?
9. Чем `thenValue(T -> U)` отличается от `thenValue(T -> Future<U>)` ?
10. Что делают `CollectAll` и `CollectAny` ?
11. Зачем нужны executors?
12. Как `via(cpuExecutor)` и `via(ioExecutor)` помогают в epoll-сервере?